```
Date: Wed, 30 Jan 2019 16:51:06 GMT
Location: /spaces/10
Content-Type: application/json
Transfer-Encoding: chunked
Server: Jetty(9.4.8.v20171121)
{"name":"', ''); DROP TABLE spaces; --","uri":"/spaces/10"}
```

Prepared statements in SQL eliminate the possibility of SQL injection attacks if used consistently. They also can have a performance advantage because the database can compile the query or statement once and then reuse the compiled code for many different inputs; there is no excuse not to use them. If you're using an object-relational mapper (ORM) or other abstraction layer over raw SQL commands, check the documentation to make sure that it's using prepared statements under the hood. If you're using a non-SQL database, check to see whether the database API supports parameterized calls that you can use to avoid building commands through string concatenation.

2.4.2 Mitigating SQL injection with permissions

While prepared statements should be your number one defense against SQL injection attacks, another aspect of the attack worth mentioning is that the database user didn't need to have permissions to delete tables in the first place. This is not an operation that you would ever require your API to be able to perform, so we should not have granted it the ability to do so in the first place. In the H2 database you are using, and in most databases, the user that creates a